Simulador de gestor de base de datos con organización eficiente de archivos

Alferez Vicente, Bladimir
Dept. de Data Science
UTEC
Lima, Perú
bladimir.alferez@utec.edu.pe

I. Introducción

El objetivo de este proyecto es construir un simulador de gestor de base de datos que permita entender cómo se organizan y acceden los datos en memoria secundaria. En lugar de usar un motor de base de datos ya existente, se implementó una solución propia en Python, donde los registros se almacenan en archivos binarios y todas las operaciones de lectura y escritura se realizan mediante páginas de tamaño fijo. Esto permite observar de forma más clara el costo real de cada operación, tanto en accesos a disco como en tiempo de ejecución.

Para realizar las pruebas del sistema se utilizó el dataset *NYPD Arrest Data (Year to Date)*, obtenido desde Kaggle. Este conjunto de datos contiene información sobre arrestos registrados por el Departamento de Policía de Nueva York, incluyendo atributos como identificador del arresto, fecha, código del delito, descripción del delito, distrito, edad, sexo, raza y coordenadas geográficas. A partir de este dataset se generaron subconjuntos de distinto tamaño para evaluar el comportamiento de las estructuras de indexación implementadas. La fuente utilizada fue: https://www.kaggle.com/datasets/amirhosseinzinati/nypd-arrest-data-year-to-date.

El sistema desarrollado incluye distintas estructuras de indexación vistas en el curso, como Sequential File, Extendible Hashing, B+ Tree y R-Tree. Cada una fue implementada con un propósito distinto: Sequential File permite trabajar con archivos ordenados y un área auxiliar de desbordamiento; Extendible Hashing está pensado para búsquedas exactas; B+ Tree permite búsquedas puntuales y por rango; y R-Tree permite realizar consultas espaciales usando coordenadas de longitud y latitud.

Además, se implementó un parser para un subconjunto de SQL. Este parser permite ejecutar sentencias como `CREATE TABLE`, `SELECT`, `INSERT` y `DELETE`, y traduce cada consulta a una operación interna del índice correspondiente. De esta manera, el usuario puede interactuar con el sistema usando una sintaxis parecida a SQL, mientras que internamente el motor utiliza las estructuras implementadas para resolver cada operación.

El proyecto también cuenta con una interfaz web que permite cargar archivos CSV, escribir consultas SQL, visualizar los resultados en una tabla y revisar métricas como lecturas de páginas, escrituras de páginas y tiempo de ejecución. Para las consultas espaciales, la interfaz muestra una visualización gráfica de los puntos consultados mediante el R-Tree. Asimismo, se agregó un simulador básico de concurrencia para mostrar cómo dos transacciones pueden acceder a las mismas páginas y generar posibles conflictos.

Finalmente, se realiza una evaluación experimental usando los datos cargados desde archivos CSV. En esta evaluación se comparan las estructuras implementadas considerando el número de accesos a disco y el tiempo de ejecución para diferentes tamaños de datos. Con ello, se busca analizar si el comportamiento observado en las pruebas se relaciona con lo esperado teóricamente para cada técnica de organización de archivos.

II. Técnicas utilizadas

En esta parte se describen las técnicas de indexación implementadas en el simulador de base de datos. Todas las estructuras trabajan sobre archivos almacenados en memoria secundaria y acceden a la información mediante páginas de tamaño fijo de 4096 bytes. Para ello, se implementó una capa común de almacenamiento formada por el `PageManager` y el `RecordManager`.

El `PageManager` se encarga de leer y escribir páginas completas desde disco mediante operaciones como `read_page` y `write_page`. El `RecordManager` administra los registros dentro de cada página y asigna a cada registro un identificador físico llamado `RID`, compuesto por `page_id` y `slot_id`. De esta manera, los índices no almacenan toda la fila, sino una relación entre la clave indexada y el `RID` del registro.

$$\texttt{clave} \rightarrow \texttt{RID(page_id, slot_id)}$$

Cuando una consulta encuentra una clave en un índice, el sistema usa el `RID` para leer la página correspondiente del archivo de datos y recuperar el registro completo. Esto permite separar el archivo principal de datos de las estructuras de indexación.

A. Sequential File

Sequential File fue implementado usando un archivo principal ordenado por clave y un archivo auxiliar de desbordamiento. Esta técnica permite búsquedas puntuales y

búsquedas por rango, ya que mantiene un orden lógico entre las claves.

En la inserción, primero se crea una entrada con la clave y el `RID` del registro. Si la entrada no puede colocarse directamente en el archivo principal sin reordenar físicamente el archivo, se inserta en el archivo auxiliar de desbordamiento. Cuando el auxiliar alcanza un límite definido K, se realiza una reconstrucción física: se leen las entradas del archivo principal y del auxiliar, se ordenan por clave y se reescribe el archivo principal.

En la búsqueda puntual, el sistema recorre el archivo principal hasta encontrar la clave o hasta detectar que la clave ya no podría aparecer por el orden del archivo. Si existen registros en el área auxiliar, también se revisan mediante los punteros correspondientes. En la búsqueda por rango, se ubica la primera clave dentro del intervalo y se continúa recorriendo secuencialmente hasta superar el límite superior.

En la eliminación, se busca la entrada asociada a la clave y al `RID`. Si se encuentra, se marca como eliminada o se retira del índice, actualizando los punteros lógicos si la entrada pertenece al área de desbordamiento.

B. Extendible Hashing

Extendible Hashing fue implementado para búsquedas puntuales. Esta técnica usa un directorio dinámico y buckets almacenados en páginas. Cada bucket contiene entradas de tipo clave-`RID`. A diferencia de Sequential File y B+ Tree, esta estructura no mantiene las claves ordenadas, por lo que no soporta búsquedas por rango.

En la inserción, se aplica una función hash sobre la clave. Con los bits del hash se accede a una posición del directorio, la cual apunta a un bucket. Si el bucket tiene espacio, se inserta la entrada. Si el bucket está lleno, se realiza un split. Si la profundidad local del bucket es igual a la profundidad global, primero se duplica el directorio y luego se redistribuyen las entradas entre los buckets.

En la búsqueda puntual, se calcula el hash de la clave, se accede al bucket correspondiente y se revisan sus entradas hasta encontrar la clave buscada. Esta operación suele requerir pocos accesos a disco, ya que no necesita recorrer toda la estructura.

En la eliminación, se calcula el hash de la clave, se accede al bucket correspondiente y se elimina la entrada asociada. En esta implementación no se realizó reducción del directorio después de eliminar, ya que se priorizó el funcionamiento principal de inserción, búsqueda y split de buckets.

C. B+ Tree

B+ Tree fue implementado como una estructura balanceada de nodos internos y nodos hoja. Cada nodo se almacena en una página de tamaño fijo. Los nodos internos contienen claves guía y punteros a páginas hijas, mientras que las hojas contienen las claves reales junto con los `RID`. Además, las hojas están enlazadas entre sí para facilitar las búsquedas por rango.

En la inserción, el sistema empieza desde la raíz y desciende hasta la hoja donde debe ubicarse la clave. Si la hoja tiene espacio, la entrada se inserta manteniendo el orden. Si la hoja se llena, se divide en dos hojas y se promueve una clave al nodo padre. Si el padre también se llena, el proceso de división puede propagarse hacia arriba. Si la raíz se divide, se crea una nueva raíz.

En la búsqueda puntual, se parte desde la raíz y se desciende por los nodos internos comparando la clave con las claves guía. Al llegar a la hoja correspondiente, se busca la clave y se devuelven los `RID` asociados.

En la búsqueda por rango, primero se ubica la hoja donde debería encontrarse la clave inicial. Luego, se recorren las entradas de esa hoja y de las hojas siguientes usando los punteros enlazados, hasta superar el límite superior del rango.

En la eliminación, se ubica la hoja correspondiente y se retira la entrada asociada a la clave y al `RID`. Se mantiene la estructura de hojas enlazadas para que las búsquedas por rango sigan funcionando correctamente.

D. R-Tree

R-Tree fue implementado para consultas espaciales usando las coordenadas de longitud y latitud del dataset. Esta estructura organiza los datos mediante rectángulos mínimos envolventes, conocidos como *Minimum Bounding Rectangles* o MBR. En el caso de un punto, el MBR se forma usando la misma coordenada como límite mínimo y máximo.

Cada entrada del R-Tree almacena un MBR y un puntero. En los nodos hoja, el puntero corresponde al `RID` del registro. En los nodos internos, el puntero corresponde a la página de un nodo hijo. Al igual que en las demás estructuras, los nodos del R-Tree se guardan en páginas de tamaño fijo.

En la inserción, el sistema busca la hoja cuyo MBR requiere menor crecimiento para incluir el nuevo punto. Luego inserta la entrada con el MBR y el `RID`. Si el nodo se llena, se realiza una división y se actualizan los MBR en los nodos superiores.

En la búsqueda por radio, el sistema revisa los nodos cuyos MBR intersectan la región de búsqueda. Al llegar a las hojas, calcula la distancia real entre el punto consultado y los puntos almacenados. Solo se devuelven los registros que se encuentran dentro del radio indicado.

Para la consulta `kNN`, el sistema busca los k puntos más cercanos al punto de consulta. Se comparan distancias y se devuelven los `RID` asociados a los puntos más cercanos.

E. Manejo de memoria secundaria

El manejo de memoria secundaria se realizó mediante páginas de tamaño fijo de 4096 bytes. Ninguna estructura accede directamente al archivo completo, sino que todas las operaciones de lectura y escritura pasan por el `PageManager`. Esto permite contabilizar cuántas páginas se leen y cuántas páginas se escriben por cada operación.

Las tablas almacenan los registros completos en archivos de datos. Los índices almacenan únicamente claves y `RID`. Por ejemplo, si una consulta busca una clave en un B+ Tree, primero se accede al índice para obtener el `RID`. Luego, con

ese `RID`, se lee la página del archivo de datos donde se encuentra el registro completo.

Este diseño permite medir el costo real de las operaciones en memoria secundaria. Para cada consulta ejecutada, el sistema registra:

- cantidad de páginas leídas;
- cantidad de páginas escritas;
- total de accesos a disco;
- tiempo de ejecución en milisegundos.

F. Comparación de accesos a memoria secundaria

La comparación entre las técnicas se realizó considerando la cantidad de páginas leídas y escritas durante las operaciones de inserción, búsqueda y eliminación. La siguiente tabla resume el comportamiento esperado de cada técnica.

Técnica	Inserción	Búsq. puntual	Eliminación
Sequential File	Alta si reconstruye	Media/alta	Media/alta
Extendible Hashing	Baja, salvo split	Baja	Baja
B+ Tree	Media	Baja/media	Media
R-Tree	Media	Media espacial	Media

TABLE I
COMPARACIÓN ESPERADA DE ACCESOS A MEMORIA SECUNDARIA.

En Sequential File, la búsqueda puede requerir revisar varias páginas del archivo principal y del archivo auxiliar. Además, la inserción puede ser costosa cuando el área de desbordamiento alcanza el límite K, porque se realiza una reconstrucción física.

En Extendible Hashing, la búsqueda puntual suele requerir pocos accesos, ya que el hash permite ubicar directamente el bucket correspondiente. La inserción también es eficiente mientras el bucket tenga espacio. Cuando el bucket se llena, se requiere un split y pueden aumentar las escrituras.

En B+ Tree, las búsquedas requieren recorrer páginas desde la raíz hasta la hoja correspondiente. Para búsquedas por rango, se aprovechan los enlaces entre hojas, por lo que no es necesario reiniciar la búsqueda desde la raíz para cada clave del intervalo.

En R-Tree, los accesos dependen de la distribución espacial de los datos. Si los MBR están bien distribuidos, se visitan menos páginas. Si hay mucho solapamiento entre MBR, el sistema debe revisar más nodos.

G. Parser SQL

Se implementó un parser propio para un subconjunto de SQL. El objetivo fue permitir que el usuario ejecute consultas con una sintaxis similar a SQL, pero sin usar motores externos de base de datos.

El parser se divide en dos etapas. Primero, el `Tokenizer` recibe la consulta como texto y la divide en tokens. Estos tokens pueden ser palabras reservadas, identificadores, números, cadenas de texto o símbolos. Por ejemplo, reconoce palabras como `CREATE`, `TABLE`, `SELECT`, `WHERE`, `INSERT`, `DELETE`, `BETWEEN`, `POINT`, `RADIUS` y `K`.

Luego, el `SQLParser` analiza la secuencia de tokens y construye un comando interno. Este comando no ejecuta la operación directamente, sino que representa la acción que debe realizar el motor. Finalmente, el `QueryExecutor` recibe ese comando y lo traduce a una llamada sobre el índice correspondiente.

Las principales traducciones realizadas son:

Sentencia SQL	Operación interna
`CREATE TABLE ... FROM FILE`	Carga CSV
`SELECT ... WHERE col = valor`	`search`
`SELECT ... BETWEEN a AND b`	`rangeSearch`
`INSERT INTO ... VALUES (...)`	`add`
`DELETE FROM ... WHERE col = valor`	`remove`
`POINT(...), RADIUS r`	`RTree.rangeSearch`
`POINT(...), K k`	`RTree.kNN`

TABLE II
TRADUCCIÓN DE SENTENCIAS SQL A OPERACIONES INTERNAS.

El parser soporta creación de tablas desde CSV, búsquedas puntuales, búsquedas por rango, inserciones, eliminaciones y consultas espaciales. También se agregó `SELECT TOP n * FROM tabla` para visualizar una cantidad limitada de registros sin cargar toda la tabla en memoria.

III. SIMULADOR DE ACCESO CONCURRENTE

Se implementó un simulador básico de concurrencia para mostrar cómo dos transacciones pueden acceder de forma intercalada a las mismas páginas de datos e índices. El objetivo no fue implementar un sistema completo de bloqueos, sino registrar el orden de ejecución de las operaciones y detectar conflictos simples cuando dos transacciones acceden al mismo recurso y al menos una realiza una operación de escritura.

Para la demostración se utilizó el archivo `nypd_arrests_1000.csv`. Primero se creó una tabla llamada `nypd_concurrency_demo` con un índice B+ Tree sobre la columna `arrest_key`. Luego se seleccionó una clave real del dataset para simular dos transacciones sobre el mismo registro.

Elemento	Valor
Tabla usada	`nypd_concurrency_demo`
Registros cargados	1000
Clave usada	238548379
RID encontrado	`RID(page=35, slot=10)`
Página de datos	35
Página de índice	0
Lecturas en carga	76814
Escrituras en carga	2405
Tiempo de carga	1736.17 ms

TABLE III
DATOS USADOS EN LA DEMO DE CONCURRENCIA.

El schedule intercalado ejecutado fue el siguiente:

A partir del log anterior, el simulador detectó tres conflictos. Estos conflictos aparecen cuando dos transacciones acceden a la misma página y una de ellas realiza una operación de escritura o eliminación.

El primer conflicto ocurre porque `T1` lee la página 35 de la tabla y luego `T2` elimina un registro ubicado en esa misma

Orden	Tx	Operación	Recurso	Página
1	T1	READ	índice B+ Tree	0
2	T2	READ	índice B+ Tree	0
3	T1	READ	tabla	35
4	T2	READ	tabla	35
5	T2	DELETE	tabla	35
6	T2	WRITE	índice B+ Tree	0
7	T1	READ	tabla	35
8	T1	COMMIT	transacción	–
9	T2	COMMIT	transacción	–

TABLE IV
SCHEDULE INTERCALADO DE LAS TRANSACCIONES.

Nro.	Tipo	Recurso afectado
1	READ-WRITE	`tabla: page 35`
2	READ-WRITE	`índice B+ Tree: page 0`
3	WRITE-READ	`tabla: page 35`

TABLE V
CONFLICTOS DETECTADOS POR EL SIMULADOR.

página. El segundo conflicto ocurre sobre la página 0 del índice B+ Tree, ya que `T1` estaba leyendo el índice mientras `T2` lo modifica como consecuencia del `DELETE`. El tercer conflicto ocurre cuando `T1` intenta volver a leer la página 35 después de que `T2` ya la modificó.

En total, el simulador registró 9 operaciones y detectó 3 conflictos. Con esto se evidencia el orden de ejecución de las transacciones y los problemas que pueden aparecer cuando varias transacciones acceden a las mismas páginas sin un mecanismo avanzado de control de concurrencia.

IV. EVALUACIÓN EXPERIMENTAL

Para la evaluación experimental se usaron subconjuntos del dataset NYPD Arrest Data con tamaños de 1000, 2500 y 5000 registros. Se compararon las técnicas B+ Tree, Extendible Hashing y Sequential File en tres operaciones principales: inserción, búsqueda puntual y búsqueda por rango. Las métricas evaluadas fueron el número promedio de accesos a disco, calculado como lecturas más escrituras, y el tiempo promedio de ejecución en milisegundos.

A. Accesos a disco

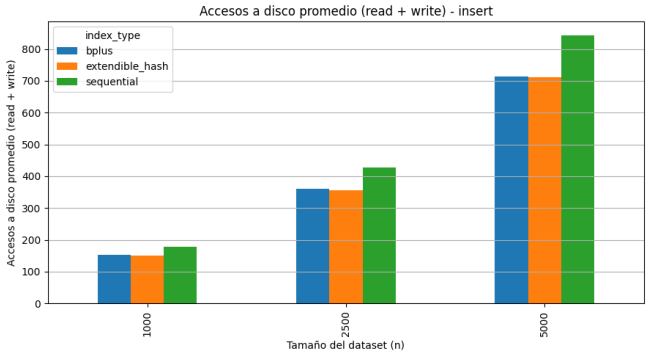

Fig. 1. Accesos a disco promedio para la operación de inserción.

//bar_io_search.png

Fig. 2. Accesos a disco promedio para la búsqueda puntual.

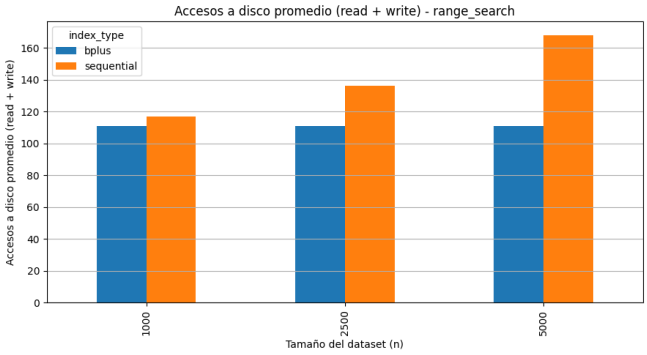

Fig. 3. Accesos a disco promedio para la búsqueda por rango.

B. Tiempo de ejecución

C. Análisis de resultados

En la operación de inserción se observa que Sequential File genera más accesos a disco que B+ Tree y Extendible Hashing. Esto se debe a que Sequential File mantiene un archivo principal ordenado y un área auxiliar de desbordamiento, por lo que algunas inserciones pueden implicar más lecturas, escrituras o reconstrucciones. En cambio, B+ Tree y Extendible Hashing presentan un comportamiento más estable para esta operación.

En la búsqueda puntual, Extendible Hashing obtuvo la menor cantidad de accesos a disco y menor tiempo de ejecución. Esto era esperado, ya que esta técnica calcula el hash de la clave y accede directamente al bucket correspondiente. B+ Tree también mantiene un bajo costo, ya que solo debe recorrer el árbol desde la raíz hasta la hoja. Sequential File fue el más costoso, porque puede requerir revisar más páginas del archivo principal y del área de desbordamiento.

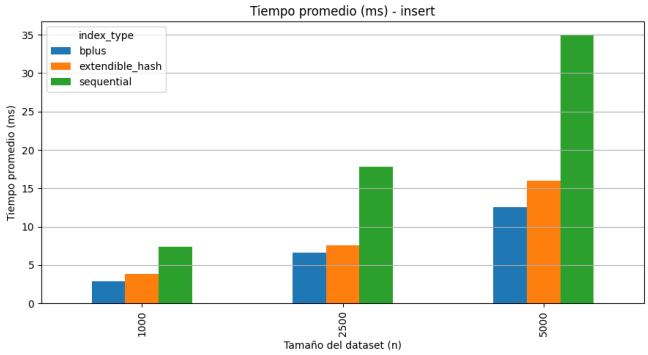

Fig. 4. Tiempo promedio de ejecución para la operación de inserción.

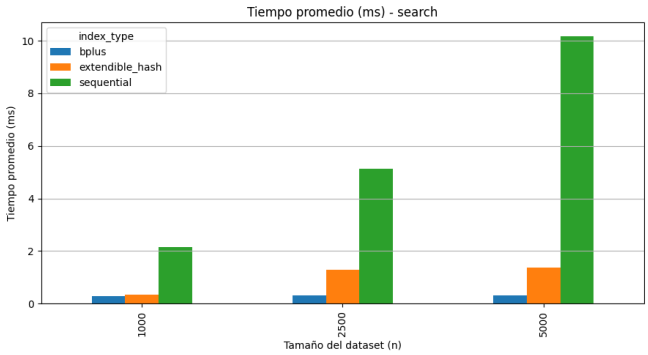

Fig. 5. Tiempo promedio de ejecución para la búsqueda puntual.

En la búsqueda por rango solo se compararon B+ Tree y Sequential File, ya que Extendible Hashing no soporta este tipo de operación. B+ Tree mostró mejores resultados porque sus hojas están ordenadas y enlazadas, lo que permite recorrer el intervalo de manera eficiente. Sequential File también puede realizar búsquedas por rango, pero el costo aumenta cuando debe revisar más páginas o considerar registros en el área auxiliar.

En general, los resultados muestran que Extendible Hashing es más conveniente para búsquedas exactas, B+ Tree es más equilibrado porque soporta búsqueda puntual y por rango, y Sequential File resulta más sensible al crecimiento del dataset debido al manejo del archivo auxiliar de desbordamiento.

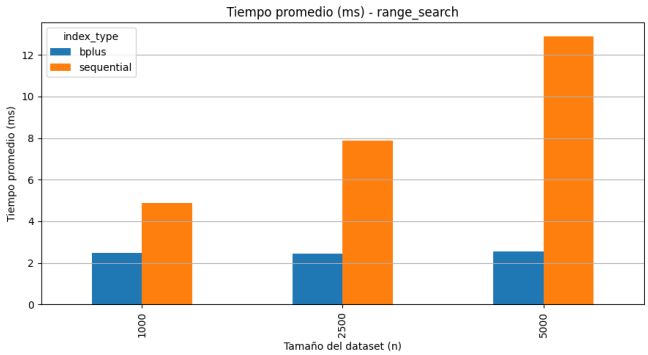

Fig. 6. Tiempo promedio de ejecución para la búsqueda por rango.